

**Curso Superior de Tecnologia em Análise e Desenvolvimento de
Sistemas**

Disciplina: Estrutura de Dados II



Docente: Everaldo Graciliano Souza Ribeiro

Ano letivo: 2026 · 2º semestre

Apostila do Projeto Final

LogiRota MVP - Planejamento e Entregas de 6 Semanas

Documento único que orienta todo o trabalho do bimestre: da persistência de dados à apresentação final do produto mínimo viável do projeto LogiRota, em 6 semanas de oficina de engenharia individual.

Período de execução: 07 de outubro a 26 de novembro de 2026.

1. Visão Geral do Projeto Final

Este documento organiza as últimas seis semanas de Estrutura de Dados II em torno de um único objetivo: transformar o LogiRota - construído incrementalmente desde a Aula 01 até a Aula 08 (TADs, listas, pilhas, filas, árvores, grafos e caminho mínimo com Dijkstra) - num produto mínimo viável (MVP) completo, individual e apresentável. Substitui, de forma elaborada, o que o plano de ensino já previa para o restante do semestre (Módulo 4 - busca, ordenação, Big-O e avaliação de desempenho - e a integração final numa interface gráfica): em vez de aulas de conteúdo isoladas, cada conceito é aplicado diretamente ao MVP, na própria semana em que faz falta.

O formato segue literalmente o que o plano de ensino da disciplina já define como "oficina de engenharia": 1 tempo de exposição para 3 tempos de construção e revisão, repetido a cada semana, mas agora com acompanhamento individual - cada aluno constrói sua própria versão do MVP, no seu próprio ritmo dentro da semana, com o professor circulando e orientando aluno por aluno nos três tempos de laboratório.

Por que um projeto individual, e por que agora

Até a Aula 08, o LogiRota foi construído coletivamente, em ritmo de aula ao vivo, com todos os alunos chegando ao mesmo código ao final de cada encontro. As seis semanas finais invertem esse eixo: a base de código (TADs, grafo, Dijkstra, visualização de mapa) já é comum e está pronta - o que resta é decisão de projeto, e decisão de projeto não se faz em coro. Cada aluno escolhe sua própria camada de apresentação, seu próprio refinamento de custo, seu próprio ritmo de implementação dentro da semana - e é avaliado por isso.

Observação: Nada do que já foi construído até a Aula 08 é descartado. Todo TAD, toda classe e todo algoritmo citados neste documento (Ponto, GrafoMatriz, GrafoLista, FilaDePedidos, PilhaDeOperacoes, dijkstra, reconstruir_caminho, mostrar_mapa) já existem no projeto e devem ser reaproveitados sem reescrita - a mesma regra de ouro seguida desde a Aula 01.

O que já existe (ponto de partida, Aula 08)

Camada	O que já está pronto
Domínio (TADs)	Ponto (nome, bairro, x, y, distancia_ate); GrafoMatriz e GrafoLista (ambas com adicionar_rua, vizinhos, vizinhos_com_peso); FilaDePedidos (FIFO) e PilhaDeOperacoes (LIFO).
Algoritmos	bfs, dfs, componentes_conexos (Aula 07); dijkstra e reconstruir_caminho, caminho mínimo entre dois pontos (Aula 08).
Apresentação	visualizacao.py - mostrar_mapa(pontos, ruas, rota=None, destaque=None, titulo=...), uma janela tkinter que desenha a

Camada	O que já está pronto
	malha e, se houver, destaca uma rota já calculada. Nunca decide nada, só desenha o que já foi calculado.
Persistência	Nenhuma ainda - todo dado vive em memória e se perde ao fechar o programa. É exatamente a lacuna que a Semana 1 fecha.

O MVP que as seis semanas constroem

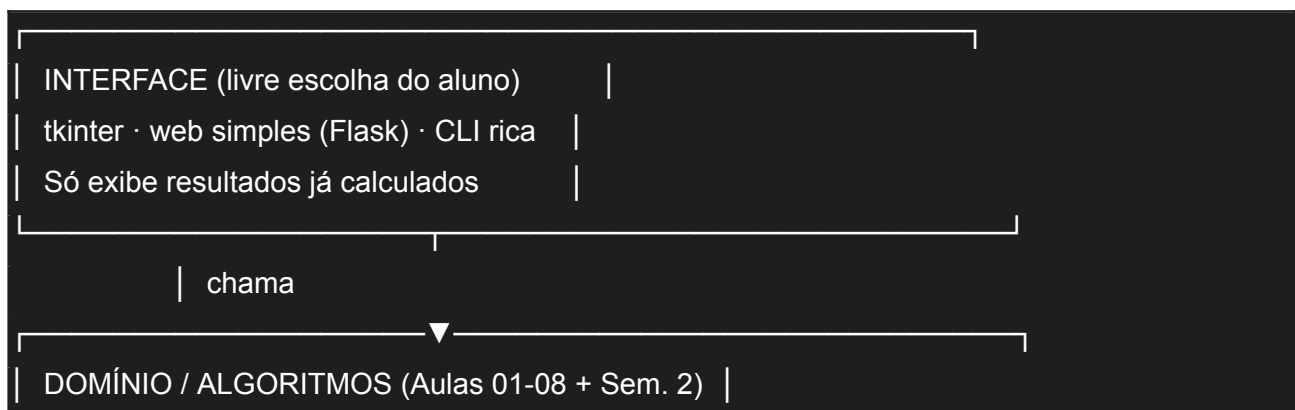
Ao final da Semana 5, cada aluno deve ter um LogiRota capaz de:

1. Cadastrar pontos de entrega e vias, e persistir esse cadastro em banco de dados - os dados sobrevivem a fechar e reabrir o programa.
2. Cadastrar pedidos de entrega (um pedido = um destino) e consultar os pedidos pendentes.
3. Calcular o caminho de menor custo entre dois pontos quaisquer (Dijkstra, já pronto).
4. Planejar uma rota otimizada para TODOS os pedidos pendentes de uma vez, numa ordem de visita que minimize a distância total percorrida - não apenas o caminho entre dois pontos, mas o roteiro de um motorista com várias paradas.
5. Exibir esse plano numa interface - tecnologia livre, mas precisa mostrar os pedidos pendentes e o resultado do planejamento de forma legível para quem não leu o código.
6. Tratar erros previsíveis (grafo desconectado, pedido sem rota possível, nenhum pedido pendente) sem travar.

"Enxuto mas funcional" é o padrão de qualidade: não se espera um sistema comercial, mas as seis funcionalidades acima precisam funcionar de ponta a ponta, com dados reais passando pelas três camadas (persistência → algoritmo → interface) sem atalho nem dado fictício escondido no código.

Arquitetura de referência

As três camadas abaixo devem permanecer separadas - a mesma separação já ensinada em `visualizacao.py` desde a Aula 08: quem desenha nunca decide, quem decide nunca desenha.



Ponto, GrafoLista, dijkstra,
planejar_rota_multipla (nova, Semana 2)

lê e grava

PERSISTÊNCIA (nova, Semana 1)
banco.py - SQLite (pontos, ruas, pedidos,
rotas_calculadas)

2. Cronograma Geral

Semana	Datas	Tema	Entrega da semana
1	07 e 08/10	Persistência e Modelagem de Dados	Banco de dados funcional: pontos, vias e pedidos sobrevivem ao fechar o programa
2	21 e 22/10	Motor de Rotas Otimizadas	planejar_rota_multipla funcionando e persistindo o resultado
3	04 e 05/11	Camada de Interface e UX	Interface mostrando pedidos pendentes e o plano de rota
4	11 e 12/11	Custo, Robustez e Desempenho	Versão refinada com tratamento de erros e métricas de custo
5	18 e 19/11	Integração Final	Projeto completo testado fim a fim, pronto para a banca
6	25 e 26/11	Apresentação Final (Banca)	Apresentação individual - 15 min de demonstração + perguntas

Formato semanal padrão (Semanas 1 a 5)

Encontro	Tempos	Conteúdo
Dia 1	1º tempo	Aula expositiva: conteúdo novo da semana, amarrado ao que já existe no projeto
Dia 1	2º tempo	Implementação acompanhada (1/3) - início individual, professor circula
Dia 2	3º tempo	Implementação acompanhada (2/3)
Dia 2	4º tempo	Implementação acompanhada (3/3) - fechamento e checagem da entrega

3. Composição da Nota do Bimestre (N2)

Conforme o plano de ensino, N2 é composta por desafios semanais de laboratório e projeto final com banca de apresentação. Este documento operacionaliza exatamente essa estrutura:

Componente	Peso	Pontos
Entrega da Semana 1 - Persistência	12%	1,2
Entrega da Semana 2 - Motor de Rotas	12%	1,2
Entrega da Semana 3 - Interface/UX	12%	1,2
Entrega da Semana 4 - Robustez e Desempenho	12%	1,2
Entrega da Semana 5 - Integração Final	12%	1,2
Apresentação Final - Banca (Semana 6)	40%	4,0
TOTAL (N2)	100%	10,0

Os quatro critérios do plano de ensino - organização do código, correção lógica, eficiência da solução escolhida e compreensão dos conceitos aplicados (incluindo justificativa de complexidade) - valem para toda entrega semanal; o peso relativo de cada um varia conforme o foco da semana, detalhado na rubrica de cada semana adiante.

Política de entregas (regra de não cumulatividade)

- Cada semana tem uma data de entrega equivalente ao segundo encontro daquela semana (ex.: Semana 1 entrega até o fim do dia 08/10).
- A etapa não entregue na data da sua própria semana vale ZERO naquele componente - não há reposição automática nem acúmulo para a semana seguinte. A nota cumulativa final do bimestre soma exatamente o que foi entregue, semana a semana.
- Entregar uma etapa atrasada, porém antes da apresentação final (Semana 6), pode ser aceito a critério do professor com desconto - não é automático, e deve ser combinado individualmente antes da próxima semana começar.
- A apresentação final (Semana 6) é obrigatória para compor qualquer nota de N2: um aluno sem apresentação não valida as entregas semanais anteriores, mesmo que todas tenham sido feitas.
- Cada entrega é estritamente individual. Reaproveitar a base comum da Aula 08 é esperado; copiar a solução de um colega para qualquer uma das seis etapas é tratado como as demais atividades da disciplina - ver convenções no Anexo B.

Semana 1 - Persistência e Modelagem de Dados

07 e 08 de outubro de 2026

Objetivos de aprendizagem

- Modelar, em tabelas relacionais simples, as entidades já existentes no domínio do LogiRota (pontos, vias) e as novas entidades que o MVP exige (pedidos de entrega, rotas calculadas).
- Entender por que um sistema de verdade não pode depender de dados vivendo só em memória, retomando a mesma motivação já vista em POO/GestorOS para persistência.
- Integrar uma camada de persistência SQLite ao projeto existente sem alterar nenhuma classe de domínio já pronta (Ponto, GrafoLista).

Aula Expositiva (1º tempo)

Roteiro de conteúdo

1. Recapitulação rápida do estado do projeto ao final da Aula 08: TADs, grafo, Dijkstra, janela de mapa - o que já existe e nunca mais deve ser reescrito do zero.
2. Apresentação do escopo do MVP final (as seis funcionalidades da Seção 1) e do cronograma das seis semanas.
3. Por que persistir: sem banco, cada execução do programa começa do zero - nenhum pedido cadastrado hoje existe amanhã. Ligação direta com o que ED1 e POO já ensinaram sobre a diferença entre dado em memória e dado salvo.
4. Modelagem das quatro tabelas do LogiRota: pontos, ruas, pedidos, rotas_calculadas - por que cada uma existe e que pergunta de negócio ela responde.
5. Por que SQLite, e não MySQL (usado em POO): projeto individual, sem servidor para configurar, arquivo único (logirota.db), biblioteca padrão do Python (sqlite3) - a mesma lógica de "enxuto mas funcional" que rege todo o MVP.
6. Apresentação do módulo banco.py, fornecido pronto - e por que: SQL e configuração de conexão não são o conteúdo desta disciplina, o contrato de uso é o que importa.

Código Fornecido - logirota/banco.py

Fornecido pronto porque a sintaxe de SQL/sqlite3 não é o foco pedagógico desta etapa - o que o aluno precisa entender é o contrato: cada função abre e fecha sua própria conexão, e devolve dados simples (tuplas, listas), sem vaziar nenhum detalhe de SQL para o resto do programa.

```
"""
Camada de persistência do LogiRota - fornecida pronta.
"""
```

```
import sqlite3

CAMINHO_BANCO = "logirota.db"

SCHEMA = """
CREATE TABLE IF NOT EXISTS pontos (
    nome TEXT PRIMARY KEY,
    bairro TEXT NOT NULL,
    x REAL NOT NULL,
    y REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS ruas (
    origem TEXT NOT NULL REFERENCES pontos(nome),
    destino TEXT NOT NULL REFERENCES pontos(nome),
    PRIMARY KEY (origem, destino)
);

CREATE TABLE IF NOT EXISTS pedidos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    destino TEXT NOT NULL REFERENCES pontos(nome),
    status TEXT NOT NULL DEFAULT 'pendente'
);

CREATE TABLE IF NOT EXISTS rotas_calculadas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    origem TEXT NOT NULL,
    sequencia TEXT NOT NULL,
    distancia_total REAL NOT NULL,
    criada_em TEXT DEFAULT CURRENT_TIMESTAMP
);
"""

def conectar():
    conexao = sqlite3.connect(CAMINHO_BANCO)
    conexao.execute("PRAGMA foreign_keys = ON")
```

```
return conexao
```

```
def inicializar():  
    """Cria as tabelas se nao existirem. Chamar uma vez, no  
    inicio do programa, antes de qualquer outra operacao."""  
    with conectar() as conexao:  
        conexao.executescript(SCHEMA)
```

logirota/banco.py - módulo completo, fornecido ao aluno no início da semana.

Aulas de Implementação Acompanhada (2º, 3º e 4º tempos)

Trabalho individual. O professor circula entre os alunos, checando os marcos abaixo na ordem - não é necessário que todos cheguem ao mesmo ponto ao mesmo tempo.

Roteiro guiado

1. Copiar banco.py para o pacote logirota/ do próprio projeto e chamar inicializar() no início de main().
2. Escrever as funções de inserção: salvar_ponto(ponto), salvar_ rua(origem, destino), salvar_pedido(destino) - cada uma monta um INSERT simples e usa with conectar() as conexao:.
3. Escrever as funções de consulta: listar_pontos(), listar_ruas(), listar_pedidos_pendentes() - cada uma devolve uma lista de tuplas via conexao.execute(...).fetchall().
4. Migrar o cadastro fixo de PONTOS e RUAS que já existe em main.py (Aula 08) para chamadas de salvar_ponto/salvar_ rua na primeira execução - só se o banco ainda estiver vazio, para não duplicar a cada vez que o programa roda.
5. Prova de fogo individual: fechar o programa, reabrir, e confirmar com o professor que os pontos/vias/pedidos cadastrados na sessão anterior continuam lá.

Perguntas de verificação (fazer a cada aluno)

- O que acontece se você rodar o programa duas vezes seguidas sem checar se o banco já tem dados? (Esperado: o aluno percebe duplicação e explica a correção - verificar antes de inserir, ou usar INSERT OR IGNORE.)
- Por que listar_pontos() devolve tuplas e não objetos Ponto direto do banco? (Esperado: o aluno entende que converter tupla→Ponto é responsabilidade da camada de domínio, não da de persistência - banco.py não deveria importar ponto.py.)

Erros comuns a observar

- Deixar uma conexão sqlite3 aberta sem fechar (fora do with) - trava o arquivo do banco em uso posterior.
- Inserir pontos repetidos a cada execução por esquecer de checar o estado atual do banco antes de popular dados fixos.
- Importar banco.py dentro de ponto.py ou grafo.py - a persistência deve ser chamada de main.py (ou de uma camada de serviço), nunca pelo próprio TAD de domínio.

Entrega da Semana 1

Observação: Formato de entrega válido para todas as seis semanas: arquivo .zip com o código completo do projeto até aquele ponto, enviado pelo canal informado pelo professor (Moodle/e-mail), com o nome do aluno no nome do arquivo.

- Código-fonte completo do projeto, incluindo logirota/banco.py integrado.
- O programa, ao rodar, lê pontos/vias/pedidos do banco de dados - não de listas fixas em código.
- Teste demonstrável: dados cadastrados numa execução aparecem na execução seguinte.

Critérios de Avaliação - Semana 1 (1,2 pontos)

Critério	Peso	Nível pleno esperado
Organização do código	0,3	banco.py separado do domínio; nenhuma classe de ponto.py/grafico.py foi alterada para acomodar persistência.
Correção lógica	0,5	Dados persistem de fato entre execuções; nenhuma duplicação ao reiniciar o programa.
Eficiência da solução	0,2	Conexões abertas e fechadas corretamente (with); nenhuma consulta redundante óbvia.
Compreensão dos conceitos	0,2	O aluno explica, quando perguntado, por que a persistência fica separada do domínio.

Semana 2 - Motor de Rotas Otimizadas

21 e 22 de outubro de 2026

Objetivos de aprendizagem

- Reconhecer a diferença entre o problema já resolvido (caminho mínimo entre DOIS pontos, Dijkstra) e o novo problema (visitar VÁRIOS pontos pendentes na ordem que minimiza a distância total).
- Implementar uma heurística gulosa (vizinho mais próximo) para decidir a ordem de visita, reaproveitando Dijkstra a cada passo sem reescrevê-lo.
- Justificar, em termos de complexidade, por que uma heurística é usada em vez de testar todas as ordens possíveis.

Aula Expositiva (1º tempo)

Roteiro de conteúdo

1. Problema motivador: um motorista tem 5 pedidos pendentes. Rodar Dijkstra da base até cada um, de forma independente, soma as cinco distâncias sem nenhuma relação entre elas - não é a rota real de um único motorista fazendo um único trajeto.
2. O problema de decidir a melhor ordem de visita entre vários pontos é uma versão do Problema do Caixeiro-Viajante (TSP): testar todas as ordens possíveis custa $O(n!)$, inviável mesmo para poucos pedidos - ligação direta com Big-O, prevista no plano de ensino.
3. Heurística do vizinho mais próximo: a cada passo, visite o pedido pendente mais perto de onde você está agora (não o mais perto da base). É gulosa - não garante o ótimo absoluto, mas é $O(n^2)$ com Dijkstra ingênuo e um resultado muito melhor do que a soma de rotas independentes.
4. Apresentação do esqueleto de `planejar_rota_multipla`, com um TODO explícito - a decisão de "qual é o vizinho mais próximo agora" é o que cada aluno implementa nos próximos três tempos.
5. Por que esse código não é fornecido pronto: decidir a ordem é exatamente o conceito novo da semana; o que é fornecido é só a moldura (assinatura, laço, acumulação de distância) para que o aluno gaste o tempo de laboratório no raciocínio, não na estrutura do código.

Código Fornecido - esqueleto de `planejar_rota_multipla`

```
from logirota.caminho import dijkstra, reconstruir_caminho
```

```

def planejar_rota_multipla(grafo, nomes, origem, pendentes):
    """Decide uma ordem de visita para os pontos em 'pendentes'
    a partir de 'origem', usando a heurística do vizinho mais
    proximo: a cada passo, vai para o pedido pendente mais perto
    de onde o motorista esta AGORA - nunca o mais perto da origem
    original.

    Reaproveita dijkstra() a cada passo - nao reescreve o
    algoritmo, so decide a ORDEM em que ele e chamado.
    """

    restantes = list(pendentes)
    posicao_atual = origem
    sequencia_completa = [origem]
    distancia_total = 0.0

    while restantes:
        distancia, predecessor = dijkstra(grafo, nomes, posicao_atual)

        # TODO: entre os pontos em 'restantes', escolha o de
        # menor valor em 'distancia' a partir de 'posicao_atual'.
        proximo = ...

        trecho = reconstruir_caminho(predecessor, posicao_atual, proximo)
        sequencia_completa.extend(trecho[1:])
        distancia_total += distancia[proximo]

        restantes.remove(proximo)
        posicao_atual = proximo

    return sequencia_completa, distancia_total

```

logirota/rotas.py (novo módulo) - esqueleto com um único TODO: a escolha do próximo ponto a visitar.

Aulas de Implementação Acompanhada (2º, 3º e 4º tempos)

Roteiro guiado

1. Implementar o TODO: escolher, dentre os restantes, o nome com menor `distancia[nome]` - um `min()` com `key`, ou um laço explícito com comparação.

2. Testar com os pedidos cadastrados na Semana 1: rodar `planejar_rota_multipla` e conferir, à mão, se a ordem devolvida realmente é a do vizinho mais próximo a cada passo (o professor pede para o aluno narrar o raciocínio passo a passo com um exemplo pequeno, 3 ou 4 pedidos).
3. Comparar o resultado da rota planejada com a soma de rotas independentes (Dijkstra chamado separadamente para cada pedido, sem considerar ordem) - o aluno deve demonstrar, com números reais do próprio banco, que agrupar entregas economiza distância.
4. Persistir o resultado: `salvar_rota_calculada(origem, sequencia, distancia_total)` em `rotas_calculadas`, reaproveitando `banco.py` da Semana 1 sem alterá-lo.

Perguntas de verificação

- O que a heurística do vizinho mais próximo pode fazer de "burro"? (Esperado: o aluno percebe que ela pode deixar um pedido distante isolado para o fim, encarecendo o último trecho - é o preço de ser gulosa, não ótima.)
- Por que não é razoável testar todas as ordens possíveis para 10 pedidos? (Esperado: cálculo de $10!$ e comparação com o tempo que isso levaria.)

Erros comuns a observar

- Rodar `dijkstra()` uma única vez no início e tentar reaproveitar as mesmas distâncias para todos os passos - errado, porque a cada parada o ponto de partida muda, e as distâncias precisam ser recalculadas a partir dali.
- Esquecer de remover o ponto escolhido de restantes, causando um laço infinito.

Entrega da Semana 2

- `planejar_rota_multipla` funcionando sobre os pedidos pendentes reais cadastrados no banco.
- Resultado (sequência e distância total) persistido em `rotas_calculadas`.
- Uma comparação numérica simples (print ou arquivo de texto) entre a rota planejada e a soma de rotas independentes, com os pedidos reais do aluno.

Critérios de Avaliação - Semana 2 (1,2 pontos)

Critério	Peso	Nível pleno esperado
Organização do código	0,2	Lógica de planejamento isolada num módulo próprio (ex.: <code>rotas.py</code>), não misturada ao <code>main()</code> .
Correção lógica	0,5	A ordem de visita de fato segue o vizinho mais próximo a cada passo, recalculando Dijkstra a partir da posição atual.

Critério	Peso	Nível pleno esperado
Eficiência / justificativa de complexidade	0,3	O aluno sabe dizer o custo aproximado da heurística ($O(n^2)$ com Dijkstra ingênuo) e por que ela foi escolhida em vez de testar todas as ordens ($O(n!)$).
Compreensão dos conceitos	0,2	Demonstra, com números reais, que a rota planejada é mais curta que a soma de rotas independentes.

Semana 3 - Camada de Interface e Experiência do Usuário

04 e 05 de novembro de 2026

Objetivos de aprendizagem

- Projetar uma interface mínima que exponha o que o usuário final do LogiRota (um despachante de entregas) precisa ver, sem expor detalhes internos do algoritmo.
- Escolher, entre três caminhos técnicos equivalentes, a tecnologia de interface mais adequada ao próprio ritmo e repertório, justificando a escolha.
- Reafirmar a separação entre decisão (domínio) e exibição (interface), já praticada desde mostrar_mapa na Aula 08.

Aula Expositiva (1º tempo)

Roteiro de conteúdo

1. O que um despachante de entregas precisa ver na tela, no mínimo: a lista de pedidos pendentes, um botão/comando para planejar a rota, e o resultado - ordem de visita e distância total, de preferência também visualmente no mapa.
2. Apresentação de três caminhos técnicos equivalentes (tabela comparativa abaixo) - a escolha é livre, mas deve ser justificada por escrito na entrega.
3. Para quem continuar com tkinter: como estender a janela de mostrar_mapa (Aula 08) com uma lista de pedidos ao lado do mapa, sem duplicar a lógica de desenho já existente em visualizacao.py.
4. Regra de ouro reafirmada: a interface escolhida NUNCA decide a rota - ela só chama planejar_rota_multipla (Semana 2) e mostra o que voltou. Se a interface contém qualquer cálculo de distância, a camada está errada.

Comparativo de tecnologias de interface (livre escolha)

Opção	Vantagem principal	Exige aprender	Recomendada para
tkinter (continuar)	Reaproveita mostrar_mapa direto, já parte pronto da Aula 08	Pouco novo - só layout de widgets adicionais	Quem quer ver o mapa de verdade, com menos risco técnico
Web simples (Flask)	Interface acessível pelo navegador, mais familiar a quem já viu web	Rotas HTTP básicas e um template HTML simples	Quem já tem alguma base web e quer esse diferencial

Opção	Vantagem principal	Exige aprender	Recomendada para
CLI rica (biblioteca rich)	Mais rápido de implementar, foco total na lógica	Pouco - é texto formatado, sem janela	

Observação: Nenhuma dessas três é "a estrutura em estudo" da disciplina - são ferramentas de interface, por isso o uso de biblioteca pronta é permitido aqui, ao contrário da regra que vale para pilhas/filas/árvores/grafos.

Código de referência - extensão opcional de visualizacao.py (quem continuar com tkinter)

```
# Dentro de mostrar_mapa, antes do mainloop():
# um Listbox ao lado do Canvas, mostrando os pedidos pendentes.

lista = tk.Listbox(janela, width=28)
lista.pack(side=tk.RIGHT, fill=tk.Y)
for _id, destino, status in listar_pedidos_pendentes():
    lista.insert(tk.END, f"#{_id} {destino} ({status})")
```

Trecho de referência, não obrigatório - ilustra onde plugar a lista de pedidos sem tocar no desenho do mapa já existente.

Aulas de Implementação Acompanhada (2º, 3º e 4º tempos)

Roteiro guiado

1. Decidir a tecnologia (se ainda não decidiu na aula expositiva) e montar o esqueleto mínimo (janela vazia, página inicial, ou menu de texto, conforme a escolha).
2. Implementar a exibição da lista de pedidos pendentes, lida de listar_pedidos_pendentes() (Semana 1) - dado real do banco, não uma lista fixa de exemplo.
3. Implementar a ação de planejar rota: um botão, uma rota HTTP, ou uma opção de menu que chama planejar_rota_multipla (Semana 2) e exibe o resultado.
4. Se optou por tkinter: integrar o resultado a mostrar_mapa, passando rota= com a sequência encontrada.

Perguntas de verificação

- Se eu adicionar um pedido novo direto no banco (fora do seu programa) e reabrir a interface, ele aparece? (Esperado: sim - prova de que a interface lê do banco, não de uma cópia em memória desatualizada.)
- Onde, no seu código, a distância é calculada? (Resposta esperada: nunca dentro do código da interface - sempre em rotas.py/caminho.py.)

Erros comuns a observar

- Calcular ou formatar distância dentro do código da interface (ex.: somar valores manualmente numa função de botão) - sinal de que a separação de camadas foi quebrada.
- Hardcodar os pedidos exibidos em vez de consultar o banco a cada abertura da tela.

Entrega da Semana 3

- Interface funcional (qualquer uma das três opções, ou outra equivalente aprovada previamente pelo professor) mostrando pedidos pendentes reais.
- Ação visível que dispara o planejamento de rota e exibe o resultado (sequência de pontos e distância total).
- Parágrafo curto (meia página) justificando a tecnologia escolhida.

Critérios de Avaliação - Semana 3 (1,2 pontos)

Critério	Peso	Nível pleno esperado
Organização do código	0,3	Interface não contém nenhum cálculo de distância ou decisão de rota - só chamadas às camadas já existentes.
Correção lógica	0,4	Pedidos exibidos e rota calculada refletem o estado real do banco, não dados fixos.
Eficiência / adequação da escolha	0,2	Tecnologia escolhida é usada de forma coerente com seu próprio paradigma (ex.: Flask não usando um loop bloqueante de terminal por engano).
Compreensão dos conceitos	0,3	Justificativa escrita da escolha tecnológica é específica ao próprio projeto, não genérica.

Semana 4 - Custo, Robustez e Desempenho

11 e 12 de novembro de 2026

Objetivos de aprendizagem

- Refinar o conceito de "custo" de uma rota além da distância pura, quando aplicável ao próprio projeto.
- Tratar casos de erro previsíveis sem travar o programa (grafo desconectado, nenhum pedido pendente, pedido sem rota possível).
- Medir empiricamente o tempo de execução da heurística de planejamento, ligando teoria de complexidade (Semana 2) a dado real.

Aula Expositiva (1º tempo)

Roteiro de conteúdo

1. Custo nem sempre é só distância: tempo estimado, prioridade de entrega, janela de horário são refinamentos possíveis - cada aluno escolhe UM para aprofundar no próprio projeto, não é obrigatório implementar todos.
2. Checklist de robustez: o que acontece no seu programa se não houver nenhum pedido pendente? Se um ponto cadastrado não tiver nenhuma rua ligando a ele? Cada "e se" vira um teste manual guiado nesta aula.
3. Avaliação empírica de desempenho: medir o tempo real de planejar_rota_multipla variando o número de pedidos pendentes (5, 10, 20...) e observar se o crescimento acompanha a expectativa $O(n^2)$ discutida na Semana 2 - o conteúdo de "avaliação empírica" do plano de ensino, aplicado ao próprio projeto em vez de um exercício isolado.
4. Apresentação do helper medir_tempo (fornecido), usado só como instrumento de medição - não faz parte da lógica do produto final.

Código Fornecido - helper de medição

```
import time

def medir_tempo(funcao, *args, **kwargs):
    """Executa 'funcao' e devolve (resultado, segundos gastos).
    Util para comparar o custo real do planejamento de rota
    sobre diferentes quantidades de pedidos pendentes."""
    inicio = time.perf_counter()
    resultado = funcao(*args, **kwargs)
```

```
fim = time.perf_counter()
return resultado, fim - inicio
```

logirota/medicao.py - instrumento de medição, não faz parte da lógica de negócio do produto final.

Aulas de Implementação Acompanhada (2º, 3º e 4º tempos)

Roteiro guiado

1. Checklist de robustez individual: o professor provoca cada "e se" ao vivo com o aluno (remover todas as ruas de um ponto, zerar os pedidos, pedir rota para um ponto inexistente) e o aluno corrige o que travar, com try/except ou validação explícita.
2. Escolher e implementar UM refinamento de custo (prioridade, tempo estimado, ou outro justificável) - opcional de qual, obrigatório ter pelo menos um.
3. Rodar medir_tempo sobre planejar_rota_multipla com conjuntos de pedidos de tamanhos crescentes (pode ser dado fictício só para esse teste) e anotar os tempos observados.
4. Escrever duas ou três frases comparando o crescimento observado com a expectativa teórica ($O(n^2)$) - não precisa de gráfico, só da comparação qualitativa.

Perguntas de verificação

- O que seu programa faz se eu pedir para planejar rota sem nenhum pedido pendente? (Esperado: mensagem clara, não um erro bruto/stack trace.)
- O tempo dobrou, quadruplicou, ou ficou igual quando você dobrou o número de pedidos? O que isso diz sobre $O(n^2)$? (Esperado: o aluno relaciona o número observado à expectativa teórica, mesmo que de forma aproximada.)

Erros comuns a observar

- Capturar toda exceção genericamente (except Exception: pass) só para o programa não travar, escondendo o erro real em vez de tratá-lo.
- Medir o tempo incluindo a abertura de conexão com o banco ou impressão em tela, poluindo a medição do algoritmo em si.

Entrega da Semana 4

- Pelo menos três cenários de erro tratados sem travar o programa (grafo desconectado, sem pedidos pendentes, destino inexistente).
- Um refinamento de custo implementado e justificado.
- Tabela ou texto curto com os tempos medidos para diferentes quantidades de pedidos, e uma frase de interpretação.

Critérios de Avaliação - Semana 4 (1,2 pontos)

Critério	Peso	Nível pleno esperado
Organização do código	0,2	Tratamento de erro específico por tipo de falha, não um bloco genérico escondendo tudo.
Correção lógica / robustez	0,4	Os três cenários de erro combinados na aula não travam o programa.
Eficiência / justificativa de complexidade	0,4	Medições reais apresentadas e comparadas, mesmo que informalmente, com a expectativa $O(n^2)$.
Compreensão dos conceitos	0,2	Sabe explicar por que escolheu o refinamento de custo implementado.

Semana 5 - Integração Final

18 e 19 de novembro de 2026

Objetivos de aprendizagem

- Garantir que as três camadas (persistência, algoritmo, interface) funcionam juntas, de ponta a ponta, sem gambiarra de última hora.
- Organizar o projeto para leitura e demonstração por terceiros - não só para quem o escreveu.
- Preparar o roteiro da própria apresentação final.

Aula Expositiva (1º tempo)

Roteiro de conteúdo

1. Checklist de integração fim a fim (ver tabela abaixo) - percorrido coletivamente, cada aluno confere o próprio projeto durante a aula.
2. Boas práticas de organização de projeto final: nomes de arquivo claros, um README mínimo explicando como rodar, remoção de código morto/comentado e de prints de depuração esquecidos.
3. Como estruturar uma demonstração ao vivo de 15 minutos: comece pelo problema (não pelo código), mostre o fluxo completo com dado real, termine com uma decisão técnica da qual você se orgulha.
4. Critérios da banca da Semana 6, apresentados com antecedência - nenhuma surpresa na avaliação final.

Checklist de Integração Fim a Fim

Verificação	Como confirmar
Banco de dados é a única fonte de verdade	Apagar logirota.db, rodar o programa, e confirmar que ele recria as tabelas sem travar
Planejamento de rota usa dados reais do banco	Cadastrar um pedido novo pela interface e replanejar - ele deve aparecer na nova rota
Interface nunca calcula distância sozinha	Ler o próprio código da interface e confirmar que toda distância vem de <code>rotas.py/caminho.py</code>
Erros previsíveis não travam o programa	Repetir os três cenários da Semana 4 uma última vez

Verificação	Como confirmar
Projeto roda do zero numa pasta limpa	Copiar o projeto para outra pasta e rodar sem depender de nenhum arquivo "esquecido" fora dela

Aulas de Implementação Acompanhada (2º, 3º e 4º tempos)

Roteiro guiado

1. Tempo de bancada livre, individual: cada aluno percorre o checklist de integração com o próprio projeto, corrigindo o que falhar.
2. Organizar a estrutura de pastas final (ver Anexo A) e escrever um README.md curto: o que o projeto faz, como instalar dependências (se houver), como rodar.
3. Escrever o roteiro da apresentação (ver modelo abaixo) e ensaiar a demonstração ao menos uma vez com o professor ou um colega assistindo.

Modelo de roteiro de apresentação (a preencher por cada aluno)

1. Contexto (1 min): qual problema o LogiRota resolve.
2. Demonstração ao vivo (8-10 min): cadastrar um pedido novo, planejar a rota, mostrar o resultado na interface escolhida.
3. Decisões técnicas (3-4 min): por que esta tecnologia de interface, por que este refinamento de custo (Semana 4).
4. Dificuldades e o que ficaria para uma próxima versão (1-2 min).

ROTEIRO_APRESENTACAO.md - modelo fornecido para cada aluno preencher com o próprio conteúdo.

Entrega da Semana 5

- Projeto completo, testado com o checklist de integração, numa pasta organizada com README.md.
- Roteiro de apresentação preenchido (ROTEIRO_APRESENTACAO.md ou equivalente).

Critérios de Avaliação - Semana 5 (1,2 pontos)

Critério	Peso	Nível pleno esperado
Organização do código	0,4	Estrutura de pastas clara, README explicando como rodar, sem código morto ou prints de depuração esquecidos.
Correção lógica	0,4	Checklist de integração passa do início ao fim, numa cópia limpa do projeto.

Critério	Peso	Nível pleno esperado
Compreensão dos conceitos	0,4	Roteiro de apresentação reflete decisões técnicas reais do próprio projeto, não um texto genérico.

Semana 6 - Apresentação Final (Banca)

25 e 26 de novembro de 2026

Observação: Esta semana não segue o formato 1+3. O primeiro tempo é expositivo (abertura e orientações finais); os demais tempos são integralmente dedicados às apresentações individuais.

Aula Expositiva de Abertura (1º tempo)

- Reapresentação dos critérios da banca (tabela abaixo), ordem de apresentação, e tempo exato por aluno.
- Lembrete das regras: apresentação individual, 15 minutos (10 de demonstração + 5 de perguntas), projeto deve ser o LogiRota finalizado conforme as seis funcionalidades da Seção 1.

Apresentações Individuais (demais tempos)

Cada aluno apresenta seu próprio LogiRota, seguindo o roteiro preparado na Semana 5: contexto, demonstração ao vivo com dado real (não dados fictícios preparados só para a demo, salvo exceção combinada), decisões técnicas, e dificuldades enfrentadas. O professor e os colegas podem perguntar livremente nos 5 minutos finais.

Critérios de Avaliação - Banca Final (4,0 pontos)

Critério	Peso	Nível pleno esperado
Funcionalidade do MVP	1,5	As seis funcionalidades da Seção 1 estão presentes e demonstráveis ao vivo, com dado real persistido.
Correção lógica e robustez	1,0	A rota planejada está correta; erros previsíveis não derrubam a demonstração.
Qualidade técnica e justificativa de complexidade	1,0	O aluno explica com segurança a heurística de rota, seu custo aproximado, e as decisões de arquitetura tomadas.
Comunicação e autoria	0,5	Apresentação clara, dentro do tempo, e o aluno demonstra domínio real do próprio código ao responder perguntas.

Observação: "Projeto final autoral" significa que a camada de interface, os refinamentos de custo e as decisões de organização são próprios de cada aluno - mas o núcleo (grafo, Dijkstra, planejamento multi-entrega, persistência) é o mesmo LogiRota construído pela turma inteira, e deve ser reconhecível como tal pela banca.

Anexo A - Estrutura de Pastas Esperada

```
logirota_projeto/
├── main.py
├── logirota.db          (gerado automaticamente)
├── README.md
├── ROTEIRO_APRESENTACAO.md
└── logirota/
    ├── __init__.py
    ├── ponto.py        (Aula 01-08, sem alteracao)
    ├── grafo.py        (Aula 06-08, sem alteracao)
    ├── fila.py / pilha.py (Aula 07, sem alteracao)
    ├── busca.py        (Aula 07, sem alteracao)
    ├── caminho.py      (Aula 08, sem alteracao)
    ├── visualizacao.py  (Aula 08, extensoes aditivas)
    ├── banco.py        (Semana 1, fornecido)
    ├── rotas.py        (Semana 2, implementado pelo aluno)
    └── medicao.py      (Semana 4, fornecido)
```

Anexo B - Convenções Já Estabelecidas no Curso

- Nomes de classes, métodos e variáveis em português, consistente com todo o LogiRota desde a Aula 01.
- Nenhuma biblioteca pronta substitui uma estrutura já estudada manualmente (pilha, fila, árvore, grafo) - a regra não se aplica a ferramentas de interface (tkinter/Flask/rich) nem a sqlite3, que são infraestrutura, não o conteúdo em avaliação.
- Código fornecido pronto (banco.py, medicao.py) existe porque sua sintaxe não é o foco pedagógico da etapa - mesma filosofia já aplicada a visualizacao.py desde a Aula 08.
- Reaproveitar, não reescrever: todo algoritmo construído até a Aula 08 (BFS, DFS, Dijkstra) deve ser chamado pelas novas funções das seis semanas, nunca duplicado.
- Entregas estritamente individuais a partir da Semana 1 - diferente do ritmo coletivo das Aulas 01 a 08.

Anexo C - Ambiente de Desenvolvimento (Windows)

Alunos usam Windows nos computadores do laboratório e, em muitos casos, em casa. Pontos já identificados em aulas anteriores do mesmo projeto (Aula 08) e que continuam valendo:

- Comando para rodar: `python main.py` - Windows não tem o alias `python3` por padrão.
- `tkinter` vem incluído por padrão no instalador oficial do Python para Windows (python.org) - normalmente não é necessário instalar nada à parte, diferente do que ocorreu no ambiente de desenvolvimento em Mac usado para preparar o curso.
- `sqlite3` também é biblioteca padrão do Python - nenhuma instalação adicional é necessária para a Semana 1.
- Se um aluno optar por Flask (Semana 3), é a única dependência externa do projeto inteiro - instalar com `pip install flask`, e declarar essa dependência no README.
- Em caso de janela `tkinter` aparecendo em branco: redimensionar a janela manualmente costuma resolver (sintoma conhecido já tratado na Aula 08); se persistir, confirmar qual interpretador Python está de fato selecionado no VS Code (ícone na barra de status).